

BRACT's
Vishwakarma Institute of Technology, Pune
Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 6

Roll no: 73 PRN No: 12415152 Name of Student: **Pranav Upare**

Subject Name : Deep Learning

Problem Statement

Apple crops are affected by several foliar diseases such as apple scab, black rot, and cedar apple rust, which can significantly reduce fruit yield and quality if not detected early. Manual identification of these diseases from leaf images is time-consuming and requires expert knowledge. This project aims to develop a Convolutional Neural Network (CNN) to automatically classify apple leaf images into three disease categories — Apple Scab, Black Rot, and Cedar Apple Rust — using the Apple Leaf Disease Symptoms dataset.

Algorithm :-

- 1 Load the Apple Leaf Disease Symptoms dataset containing 480 images across 3 disease classes: Apple_scab, Apple_black_rot, and Apple_cedar_rust.
- 2 Preprocess the images by resizing them to 128×128 pixels and normalizing pixel values to the range 0–1.
- 3 Split the dataset into training (70%), validation (15%), and testing (15%) sets.
- 4 Apply data augmentation such as rotation, zooming, shifting, and horizontal flipping to training images.
- 5 Build the CNN model using three convolutional layers followed by max-pooling layers.
- 6 Add fully connected layers with ReLU activation and dropout to reduce overfitting.
- 7 Use a 3-neuron softmax output layer to classify the images into the 3 disease categories.
- 8 Compile and train the model using the Adam optimizer and categorical cross-entropy loss for 10 epochs.
- 9 Evaluate the trained model using test accuracy, loss, precision, recall, F1-score, and a confusion matrix.
- 10 Predict new apple leaf images using the trained CNN and display the predicted disease class with its confidence score.

Analysis:-

- 1 The Apple Leaf Disease Symptoms dataset contains 480 images belonging to 3 classes: Apple_scab (150 images), Apple_cedar_rust (160 images), and Apple_black_rot (170 images). This dataset does not include a separate healthy class.
- 2 The dataset was divided into 335 training images, 72 validation images, and 73 testing images.
- 3 Data augmentation was applied to training images to improve model generalization and reduce overfitting.
- 4 The CNN consists of three convolutional layers with max-pooling, followed by dense and dropout layers, ending in a 3-neuron softmax output layer, with a total of 3,305,027 trainable parameters.
- 5 Training accuracy rose steadily to 88.96% by the final epoch, while validation accuracy fluctuated between 70.83% and 86.11%, peaking at epoch 10 with 86.11%. This fluctuation is expected given the very small validation set of only 72 images.
- 6 On the unseen test dataset, the model achieved a test accuracy of 80.82% with a test loss of 0.7915.
- 7 The gap between the best validation accuracy and the final test accuracy indicates some overfitting, which is common with a dataset of this size (480 images total) and reflects the noisy validation curve seen across epochs.
- 8 The classification report shows Apple_scab had a low recall (0.57), meaning many scab images were misclassified as another class. The confusion matrix confirms this: 9 of 23 true Apple_scab images were predicted as Apple_black_rot, which also pulled down Apple_black_rot's precision to 0.67.

- 9 Apple_cedar_rust was classified most reliably (precision 0.95, recall 0.83), likely due to its visually distinctive orange rust spots compared to the darker lesions of scab and black rot.
- 10 Overall test accuracy was 81%, with a macro-average F1-score of 0.80, showing the model learned meaningful disease-specific features despite the small dataset, though it would benefit from more training data, transfer learning, or class-balancing to reduce the scab/black-rot confusion.
- 11 This scab/black-rot confusion was confirmed on a live single-image test: an Apple_scab leaf image was predicted as Apple_black_rot with 99.24% confidence, showing the model is not just occasionally unsure between these two classes but can be confidently wrong — a useful finding for the discussion of model limitations.

OUTPUT:-

Dataset extraction & class inventory

```
class_root, class_dirs = find_class_root("/content/apple_raw")
print("Class folders found in:", class_root)
print("Classes:", class_dirs)

for cls in class_dirs:
    n = len([f for f in os.listdir(os.path.join(class_root, cls)) if
f.lower().endswith(IMAGE_EXTS)])
    print(f" {cls}: {n} images")
```

```
Class folders found in: /content/apple_raw
Classes: ['Apple_scab', 'Apple_cedar_rust', 'Apple_black_rot']
  Apple_scab: 150 images
  Apple_cedar_rust: 160 images
  Apple_black_rot: 170 images
```

Dataset split

```
Dataset splitting completed successfully!
train images: 335
validation images: 72
test images: 73
```

Model architecture

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

Total params: 3,305,027 (12.61 MB) Trainable params: 3,305,027 (12.61 MB) Non-trainable params: 0 (0.00 B)

Model training (10 epochs)

```
model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
history = model.fit(train_generator, epochs=10, validation_data=validation_generator)
```

```
Epoch 1/10   11/11 — 9s 735ms/step - accuracy: 0.3731 - loss: 1.1504 - val_accuracy: 0.3611 -
val_loss: 1.0064
Epoch 2/10   11/11 — 8s 771ms/step - accuracy: 0.5164 - loss: 0.9220 - val_accuracy: 0.5556 -
val_loss: 0.9724
Epoch 3/10   11/11 — 8s 648ms/step - accuracy: 0.5791 - loss: 0.9125 - val_accuracy: 0.7639 -
val_loss: 0.6962
```

Epoch 4/10 11/11 - 10s 702ms/step - accuracy: 0.7582 - loss: 0.7267 - val_accuracy: 0.8333 - val_loss: 0.4506
 Epoch 5/10 11/11 - 8s 700ms/step - accuracy: 0.7731 - loss: 0.5314 - val_accuracy: 0.8194 - val_loss: 0.6704
 Epoch 6/10 11/11 - 7s 628ms/step - accuracy: 0.8896 - loss: 0.3335 - val_accuracy: 0.8194 - val_loss: 0.6259
 Epoch 7/10 11/11 - 8s 700ms/step - accuracy: 0.9164 - loss: 0.2756 - val_accuracy: 0.8333 - val_loss: 0.7035
 Epoch 8/10 11/11 - 7s 617ms/step - accuracy: 0.9045 - loss: 0.3084 - val_accuracy: 0.8194 - val_loss: 0.7646
 Epoch 9/10 11/11 - 8s 741ms/step - accuracy: 0.8866 - loss: 0.3513 - val_accuracy: 0.7083 - val_loss: 1.1236
 Epoch 10/10 11/11 - 8s 729ms/step - accuracy: 0.8896 - loss: 0.3409 - val_accuracy: 0.8611 - val_loss: 0.4733

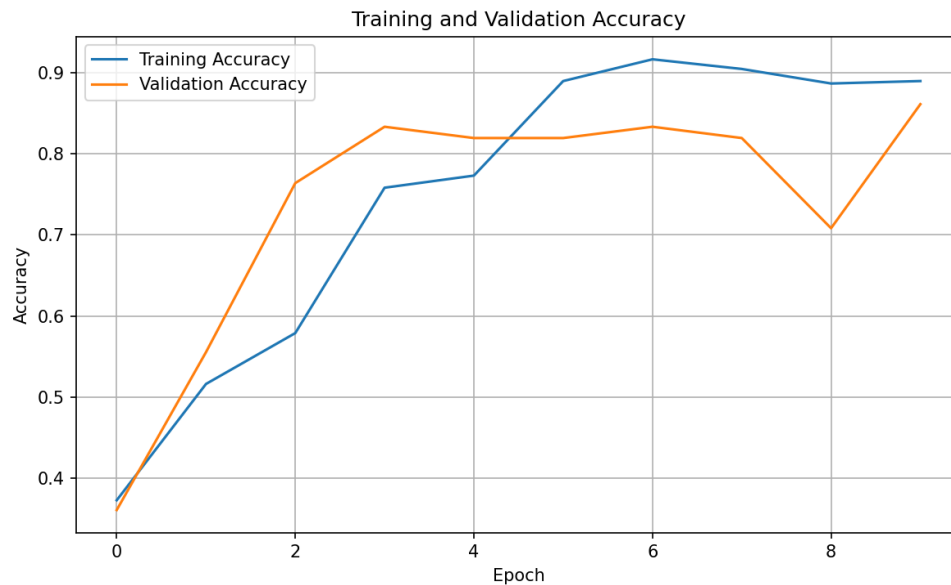


Fig 1: Training vs Validation Accuracy



Fig 2: Training vs Validation Loss

Test set evaluation

```
test_loss, test_accuracy = model.evaluate(test_generator)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

3/3 - 0s 105ms/step - accuracy: 0.8082 - loss: 0.7915

Test Loss: 0.7914817929267883
Test Accuracy: 0.8082191944122314

Classification report

	precision	recall	f1-score	support
Apple_black_rot	0.67	1.00	0.80	26
Apple_cedar_rust	0.95	0.83	0.89	24
Apple_scab	1.00	0.57	0.72	23
accuracy			0.81	73
macro avg	0.87	0.80	0.80	73
weighted avg	0.87	0.81	0.80	73

Confusion matrix

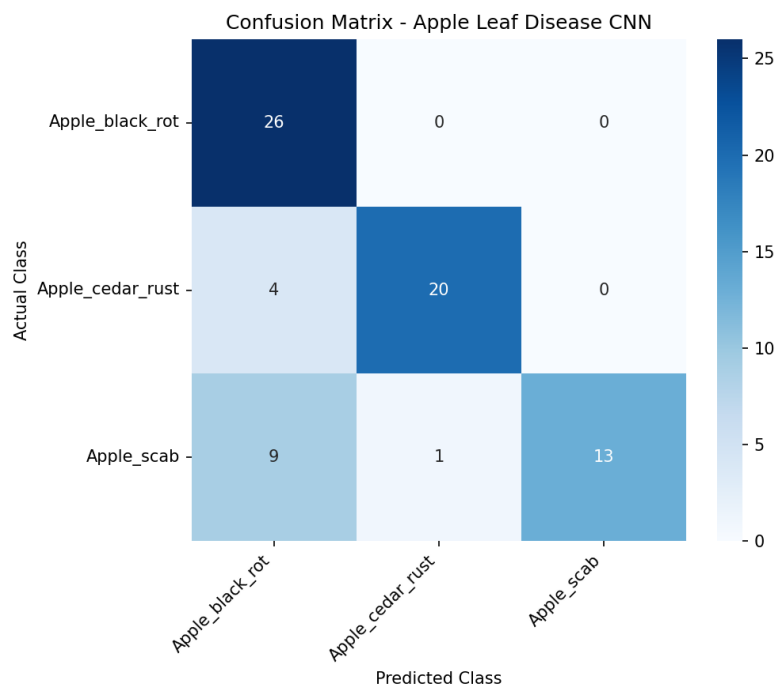


Fig 3: Confusion Matrix - Apple Leaf Disease CNN

Prediction on a new leaf image

```
img_path = "Scab (9).JPG"
img = image.load_img(img_path, target_size=(128, 128))
img_array = image.img_to_array(img) / 255.0
img_array = np.expand_dims(img_array, axis=0)
prediction = model.predict(img_array)
predicted_index = np.argmax(prediction[0])
predicted_class = class_names[predicted_index]
confidence = prediction[0][predicted_index] * 100
print("Predicted Class:", predicted_class)
print("Confidence: {:.2f}%".format(confidence))
```

1/1 - 0s 85ms/step
Predicted Class: Apple_black_rot
Confidence: 99.24%

Note: the input image was actually an Apple_scab sample (filename "Scab (9).JPG"), but the model predicted Apple_black_rot with high confidence. This is a real, observed misclassification and is consistent with the confusion matrix above, where 9 of 23 true Apple_scab test images were incorrectly predicted as Apple_black_rot — the two diseases produce visually similar dark, irregular lesions on the leaf surface, which the model struggles to separate given the limited training data.

Model saving

```
os.makedirs("/content/models", exist_ok=True)
model.save("/content/models/apple_disease_cnn.keras")
print("Model saved successfully!")
```

Model saved successfully!